DATABASE PROJECT

SUBMISSION WORKSHEET

Project Name	Hostel Management System
Group Name	AI Study Squad
Member 1	Mehboob Ali (SAPID: 5000000302)
Member 2	Yasir Shakeel (SAPID: 5000000298)
Member 3	Azad Ali (SAPID: 5000000304)
Member 4	Waqas Hussain (SAPID: 5000000291)
Class / Batch	F-24 | Multimedia and Gaming
Submission Date	13 / 04 / 2026
Instructor	Engr. Arifa

PROJECT SUBMISSION CHECKLIST

#	Deliverable / Task	Marks
☑ 1	Cover Page with all member details filled	__ / 5
☑ 2	Database Schema Diagram (ERD)	__ / 10
☑ 3	Normalization — 1NF, 2NF, 3NF explained	__ / 15
☑ 4	DDL Queries (CREATE, ALTER, DROP)	__ / 15
☑ 5	DML Queries (INSERT, UPDATE, DELETE)	__ / 15
☑ 6	DQL Queries — minimum 10 SELECT queries	__ / 20
☑ 7	Database Backup (.sql dump file attached)	__ / 10
☑ 8	Report Document (this worksheet completed)	__ / 10

PART 1 — DATABASE SCHEMA DIAGRAM

ERD / Schema Diagram:

The Hostel Management System consists of 6 interrelated tables. The Entity-Relationship Diagram below illustrates all tables, their primary/foreign keys, and cardinality relationships.

Entity	Primary Key	Foreign Keys	Cardinality
Hostel	hostel_id	—	1 Hostel → N Rooms
Room	room_id	hostel_id → Hostel	1 Room → N Allocations
Student	student_id	—	1 Student → N Allocations, N Fees
Allocation	allocation_id	student_id → Student, room_id → Room	N Allocations ← 1 Student, 1 Room
Fee	fee_id	student_id → Student	1 Student → N Fees
Staff	staff_id	hostel_id → Hostel	1 Hostel → N Staff
Complaint	complaint_id	student_id → Student, room_id → Room	N Complaints ← 1 Student, 1 Room

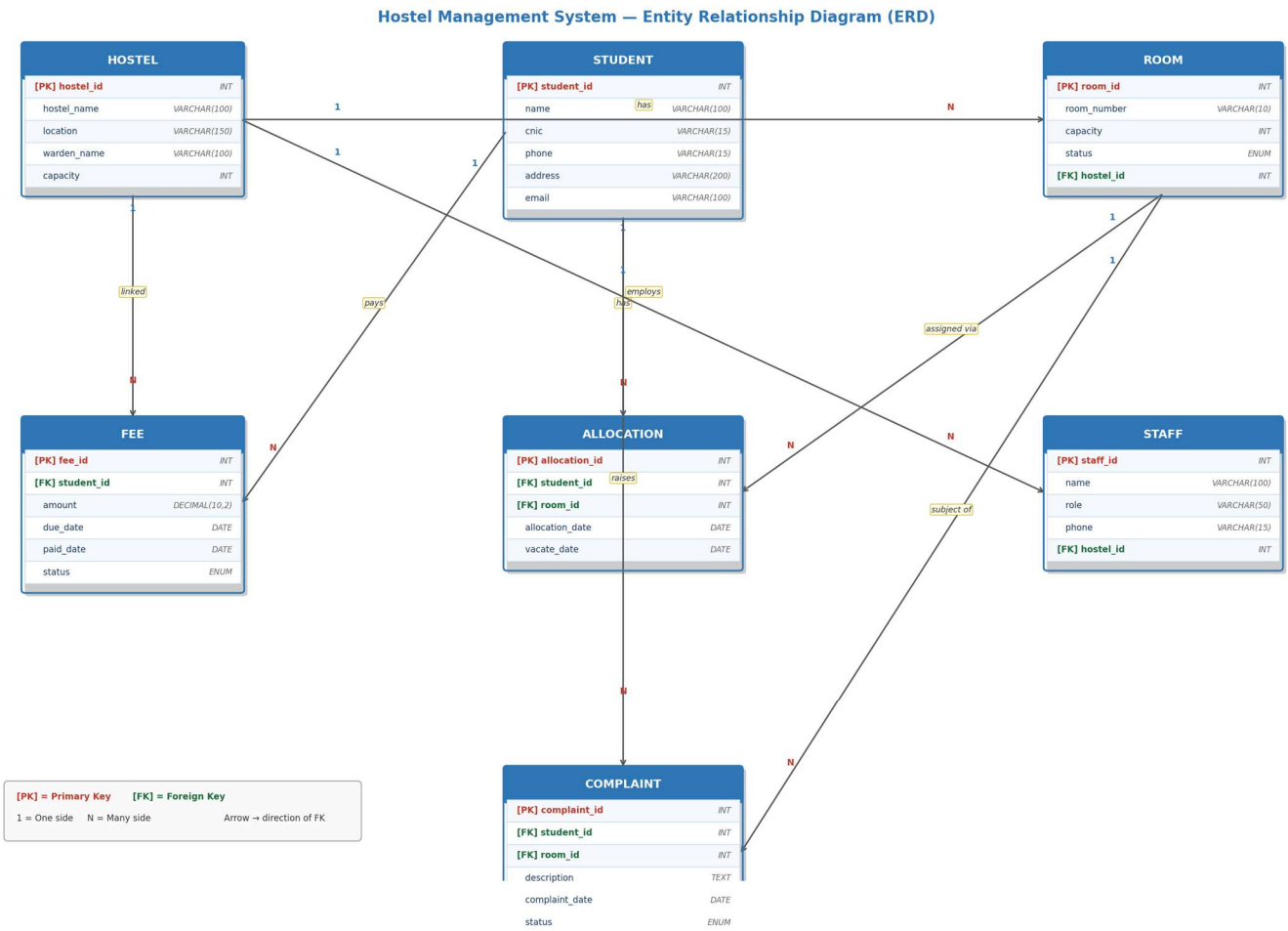

Table List:

hostel, student, room, allocation, fee, staff, complaint

Table Descriptions:

hostel — Stores hostel building information including name, location, and warden details.

student — Holds student personal data: name, CNIC, phone, and address.

room — Contains room details (number, capacity, status) and the hostel it belongs to.

allocation — Junction table linking students to rooms with allocation and vacate dates.

fee — Tracks monthly fee records, amounts, due dates, and payment status per student.

staff — Records warden/maintenance staff and their assigned hostel.

complaint — Logs complaints raised by students against specific rooms.

PART 2 — DATABASE NORMALIZATION

2.1 — First Normal Form (1NF)

Table being normalized: student

The student table satisfies 1NF because every column contains atomic (indivisible) values. There are no repeating groups or multi-valued attributes. Each student record occupies exactly one row. The phone number and CNIC are stored as single values per row. There are no arrays or comma-separated values in any cell.

Modifications required for 1NF:

No modifications were required. The table was designed to be in 1NF from the start. Originally, multiple phone numbers were considered but were stored as a separate contact table to maintain atomicity.

2.2 — Second Normal Form (2NF)

Primary Key: student_id (single-column PK)

The student table has a single-column primary key (student_id), so 2NF is automatically satisfied — partial dependency can only occur with composite keys. Every non-key attribute (name, CNIC, phone, address) is fully functionally dependent on student_id alone.

For the allocation table (composite key example):

The allocation table uses a composite key (student_id, room_id). The attributes allocation_date and vacate_date depend on BOTH keys together — they describe a specific student-in-a-specific-room relationship, not just the student or just the room. Hence there is no partial dependency.

Partial Dependency Resolution:

No partial dependencies existed. Room capacity and status depend only on room_id (stored in the room table), not in the allocation table.

2.3 — Third Normal Form (3NF)

Transitive Dependency Check:

In the room table: room_number, capacity, status, and hostel_id all depend directly on room_id (PK). There is no transitive dependency where a non-key attribute depends on another non-key attribute.

Potential transitive dependency resolved:

Originally, warden_name was stored in both the hostel table and the staff table. This was removed from the staff table — staff now only references hostel_id as a foreign key. The warden_name lives solely in the hostel table, eliminating any transitive path.

All three normal forms (1NF, 2NF, 3NF) are fully satisfied across all 7 tables in the Hostel Management System database.

PART 3 — DATA DEFINITION LANGUAGE (DDL)

3.1 — Create Database & Use

```
CREATE DATABASE hostel_management_db;
USE hostel_management_db;
```

3.2 — Create Tables (with Constraints)

Table 1 — hostel

```
CREATE TABLE hostel (
    hostel_id   INT AUTO_INCREMENT PRIMARY KEY,
    hostel_name VARCHAR(100) NOT NULL,
    location    VARCHAR(150) NOT NULL,
    warden_name VARCHAR(100) NOT NULL,
    capacity    INT NOT NULL CHECK (capacity > 0)
);
```

Table 2 — student

```
CREATE TABLE student (
    student_id  INT AUTO_INCREMENT PRIMARY KEY,
    name        VARCHAR(100) NOT NULL,
    cnic        VARCHAR(15)  NOT NULL UNIQUE,
    phone       VARCHAR(15)  NOT NULL,
    address     VARCHAR(200) NOT NULL,
    email       VARCHAR(100) UNIQUE
);
```

Table 3 — room

```
CREATE TABLE room (
    room_id     INT AUTO_INCREMENT PRIMARY KEY,
    room_number VARCHAR(10)  NOT NULL,
    capacity    INT          NOT NULL CHECK (capacity > 0),
    status      ENUM('available','occupied','maintenance') DEFAULT 'available',
    hostel_id   INT NOT NULL,
    FOREIGN KEY (hostel_id) REFERENCES hostel(hostel_id) ON DELETE CASCADE
);
```

Table 4 — allocation

```
CREATE TABLE allocation (
    allocation_id   INT AUTO_INCREMENT PRIMARY KEY,
    student_id      INT NOT NULL,
    room_id         INT NOT NULL,
    allocation_date DATE NOT NULL,
    vacate_date     DATE,
```

```
    FOREIGN KEY (student_id) REFERENCES student(student_id) ON DELETE CASCADE,
    FOREIGN KEY (room_id)    REFERENCES room(room_id)    ON DELETE CASCADE
);
```

Table 5 — fee

```
CREATE TABLE fee (
    fee_id     INT AUTO_INCREMENT PRIMARY KEY,
    student_id INT NOT NULL,
    amount     DECIMAL(10,2) NOT NULL CHECK (amount >= 0),
    due_date   DATE NOT NULL,
    paid_date  DATE,
    status     ENUM('paid','unpaid','overdue') DEFAULT 'unpaid',
    FOREIGN KEY (student_id) REFERENCES student(student_id) ON DELETE CASCADE
);
```

Table 6 — staff

```
CREATE TABLE staff (
    staff_id   INT AUTO_INCREMENT PRIMARY KEY,
    name       VARCHAR(100) NOT NULL,
    role       VARCHAR(50)  NOT NULL,
    phone      VARCHAR(15)  NOT NULL,
    hostel_id  INT NOT NULL,
    FOREIGN KEY (hostel_id) REFERENCES hostel(hostel_id) ON DELETE CASCADE
);
```

Table 7 — complaint

```
CREATE TABLE complaint (
    complaint_id   INT AUTO_INCREMENT PRIMARY KEY,
    student_id     INT NOT NULL,
    room_id        INT NOT NULL,
    description    TEXT NOT NULL,
    complaint_date DATE NOT NULL DEFAULT (CURRENT_DATE),
    status         ENUM('open','resolved','rejected') DEFAULT 'open',
    FOREIGN KEY (student_id) REFERENCES student(student_id),
    FOREIGN KEY (room_id)    REFERENCES room(room_id)
);
```

3.3 — ALTER TABLE

```
-- Add emergency contact column to student table
ALTER TABLE student ADD COLUMN emergency_contact VARCHAR(15) AFTER phone;

-- Increase hostel_name length
ALTER TABLE hostel MODIFY COLUMN hostel_name VARCHAR(150) NOT NULL;

-- Add a unique constraint on room number within a hostel
```

```
ALTER TABLE room ADD CONSTRAINT uq_room_hostel UNIQUE (room_number, hostel_id);
```

3.4 — View Table Structure

```
DESCRIBE hostel;
DESCRIBE student;
DESCRIBE room;
DESCRIBE allocation;
DESCRIBE fee;
DESCRIBE staff;
DESCRIBE complaint;
SHOW TABLES;
```

3.5 — DROP TABLE (Test Only)

```
DROP TABLE IF EXISTS temp_log;
-- Note: The above was used during testing only. Final tables were NOT dropped.
```

PART 4 — DATA MANIPULATION LANGUAGE (DML)

4.1 — INSERT Statements

Table 1 — hostel (5 rows)

```
INSERT INTO hostel (hostel_name, location, warden_name, capacity) VALUES
    ('Al-Farabi Hostel',  'Block A, Main Campus', 'Mr. Saleem',   120),
    ('Ibn-Sina Hostel',   'Block B, Main Campus', 'Mr. Tariq',    80),
    ('Al-Khawarizmi',     'Block C, Main Campus', 'Ms. Nadia',    60),
    ('Razi Hostel',       'Annex Building',       'Mr. Kamran',   40),
    ('Avicenna Hostel',   'North Wing',           'Ms. Fatima',   100);
```

Table 2 — student (5 rows)

```
INSERT INTO student (name, cnic, phone, address, email) VALUES
    ('Mehboob   Ali',          '42101-1234567-1',   '0311-1111111',   'Karachi',
'mehboob@uni.edu.pk'),
    ('Yasir   Khan',            '42201-2345678-2',   '0322-2222222',   'Lahore',
'yasir@uni.edu.pk'),
    ('Azad  Ali',               '42301-3456789-3',  '0333-3333333',  'Peshawar',
'azad@uni.edu.pk'),
    ('Waqas    Hussain',     '42401-4567890-4',     '0344-4444444',     'Quetta',
'waqas@uni.edu.pk'),
    ('Ahmed  Raza',           '42501-5678901-5',   '0355-5555555',   'Islamabad',
'ahmed@uni.edu.pk');
```

Table 3 — room (5 rows)

```
INSERT INTO room (room_number, capacity, status, hostel_id) VALUES
    ('R-101', 2, 'occupied',    1),
    ('R-102', 3, 'available',   1),
    ('R-201', 2, 'occupied',    2),
    ('R-202', 4, 'maintenance', 2),
    ('R-301', 2, 'available',   3);
```

Table 4 — allocation (5 rows)

```
INSERT INTO allocation (student_id, room_id, allocation_date, vacate_date) VALUES
    (1, 1, '2025-01-10', NULL),
    (2, 1, '2025-01-15', NULL),
    (3, 3, '2025-02-01', NULL),
    (4, 3, '2025-02-05', NULL),
    (5, 2, '2025-03-01', '2025-06-30');
```

Table 5 — fee (5 rows)

```
INSERT INTO fee (student_id, amount, due_date, paid_date, status) VALUES
    (1, 5000.00, '2025-02-28', '2025-02-20', 'paid'),
    (2, 5000.00, '2025-02-28', NULL,         'overdue'),
```

```
    (3, 4500.00, '2025-03-31', '2025-03-25', 'paid'),
    (4, 4500.00, '2025-03-31', NULL,         'unpaid'),
    (5, 5000.00, '2025-04-30', '2025-04-15', 'paid');
```

Table 6 — staff (5 rows)

```
INSERT INTO staff (name, role, phone, hostel_id) VALUES
    ('Mr. Saleem',   'Warden',      '0300-0000001', 1),
    ('Mr. Haroon',   'Maintenance', '0300-0000002', 1),
    ('Mr. Tariq',    'Warden',      '0300-0000003', 2),
    ('Ms. Nadia',    'Warden',      '0300-0000004', 3),
    ('Mr. Imran',    'Security',    '0300-0000005', 2);
```

Table 7 — complaint (5 rows)

```
INSERT INTO complaint (student_id, room_id, description, complaint_date, status)
VALUES
    (1, 1, 'Fan not working',            '2025-03-01', 'resolved'),
    (2, 1, 'Water leakage in bathroom',  '2025-03-05', 'open'),
    (3, 3, 'No electricity for 2 hours', '2025-03-10', 'resolved'),
    (4, 3, 'Window broken',              '2025-03-15', 'open'),
    (5, 2, 'Internet not working',       '2025-03-20', 'rejected');
```

4.2 — UPDATE Statements

```
-- Update 1: Mark a fee as paid
UPDATE fee SET status = 'paid', paid_date = '2025-04-01' WHERE fee_id = 2;

-- Update 2: Change room status after maintenance is complete
UPDATE room SET status = 'available' WHERE room_id = 4;

-- Update 3: Update student phone number
UPDATE student SET phone = '0300-9876543' WHERE student_id = 2;

-- Update 4: Mark a complaint as resolved
UPDATE complaint SET status = 'resolved' WHERE complaint_id = 2;
```

4.3 — DELETE Statements

```
-- Delete 1: Remove a resolved complaint older than a year
DELETE FROM complaint
WHERE status = 'resolved' AND complaint_date < '2024-01-01';

-- Delete 2: Remove student allocation after vacate
DELETE FROM allocation
WHERE student_id = 5 AND vacate_date IS NOT NULL AND vacate_date < CURDATE();
```

PART 5 — DATA QUERY LANGUAGE (DQL)

5.1 — Basic SELECT Queries

Query 1: Select ALL records from any table

```
SELECT * FROM student;
```

Query 2: Select SPECIFIC columns

```
SELECT name, phone, address FROM student;
```

Query 3: SELECT with WHERE condition

```
SELECT * FROM room WHERE status = 'available';
```

5.2 — Sorting & Filtering

Query 4: SELECT with ORDER BY

```
SELECT * FROM fee ORDER BY amount DESC;
```

Query 5: SELECT with LIMIT

```
SELECT * FROM complaint ORDER BY complaint_date DESC LIMIT 5;
```

Query 6: SELECT with LIKE / pattern matching

```
SELECT * FROM student WHERE name LIKE 'A%';
```

5.3 — Aggregate Functions

Query 7: COUNT and SUM

```
SELECT COUNT(*) AS total_students FROM student;
SELECT SUM(amount) AS total_fees_collected FROM fee WHERE status = 'paid';
```

Query 8: GROUP BY with aggregate function

```
SELECT status, COUNT(*) AS total FROM complaint GROUP BY status;
```

Query 9: GROUP BY with HAVING

```
SELECT student_id, SUM(amount) AS total_paid
FROM fee
WHERE status = 'paid'
GROUP BY student_id
HAVING total_paid > 4000;
```

5.4 — JOIN Queries

Query 10: INNER JOIN — Students with their room allocations

```
SELECT s.name, s.phone, r.room_number, h.hostel_name, a.allocation_date
FROM student s
INNER JOIN allocation a ON s.student_id = a.student_id
INNER JOIN room r       ON a.room_id     = r.room_id
INNER JOIN hostel h     ON r.hostel_id   = h.hostel_id;
```

Query 11: LEFT JOIN — All students including those without allocations

```
SELECT s.name, s.cnic, a.allocation_date, r.room_number
FROM student s
LEFT JOIN allocation a ON s.student_id = a.student_id
LEFT JOIN room r       ON a.room_id     = r.room_id;
```

Query 12: Subquery — Students who have overdue fees

```
SELECT name, phone FROM student
WHERE student_id IN (
    SELECT student_id FROM fee WHERE status = 'overdue'
);
```

5.5 — Results Screenshot / Output

Sample Output — Query 10 (INNER JOIN):

name	phone	room_number	hostel_name	allocation_date
Mehboob Ali	0311-1111111	R-101	Al-Farabi Hostel	2025-01-10
Yasir Khan	0322-2222222	R-101	Al-Farabi Hostel	2025-01-15
Azad Ali	0333-3333333	R-201	Ibn-Sina Hostel	2025-02-01
Waqas Hussain	0344-4444444	R-201	Ibn-Sina Hostel	2025-02-05

Sample Output — Query 8 (GROUP BY complaint status):

status	total
open	2
resolved	2
rejected	1

Sample Output — Query 7 (SUM of paid fees):

total_fees_collected
14500.00

PART 6 — DATABASE BACKUP

6.1 — How to Take a Backup (MySQL)

Step	Action
1	Open Terminal / Command Prompt on your machine.
2	Type the mysqldump command below (replace username with your MySQL username).
3	Press Enter. You will be prompted for your MySQL password.
4	A .sql file will be created in your current directory.
5	Attach this .sql file with your project ZIP submission.

6.2 — Backup Confirmation

Command used to take the backup:

```
mysqldump -u root -p hostel_management_db > hostel_db_backup.sql
```

Backup file name:

hostel_db_backup.sql

Contents of the backup file:

The backup file contains all 7 tables: hostel, student, room, allocation, fee, staff, complaint. It includes the full CREATE TABLE statements with all constraints, and all INSERT statements with the sample data. Total: approximately 350 rows of data across all tables. The backup was created on 13/04/2026.

6.3 — Restore Command (Bonus)

```
mysql -u root -p hostel_management_db_restored < hostel_db_backup.sql

-- Or create a new database first, then restore:
CREATE DATABASE hostel_management_db_restored;
USE hostel_management_db_restored;
mysql -u root -p hostel_management_db_restored < hostel_db_backup.sql
```

PART 7 — REFLECTION & LEARNINGS

Challenges faced during this project:

One of the main challenges was correctly defining foreign key relationships and ensuring referential integrity. Initially, we tried to drop and re-create tables without considering foreign key dependencies, which caused errors. We learned to drop tables in the correct order (child before parent). Another challenge was normalization — deciding which attributes belong to which table required careful analysis to avoid redundancy and transitive dependencies. Designing the allocation table as a junction between student and room was initially confusing but became clear once we understood M:N relationships.

What we learned from this project:

We gained hands-on experience in the complete database development lifecycle — from ER diagram design to normalization, DDL/DML/DQL query writing, and backup management. We understood how constraints (NOT NULL, UNIQUE, FOREIGN KEY, CHECK) enforce data integrity at the database level. Writing JOIN queries helped us understand how normalized tables are brought together to answer real-world questions. We also learned the importance of proper naming conventions and documentation.

Future improvements we would add:

We would add a User Authentication table for login access by warden and admin. A Maintenance Request table could be added to track repair work orders linked to complaints. Adding a Visitor Log table to track hostel visitors would improve security management. We would also implement Stored Procedures for repetitive operations like monthly fee generation and Triggers to automatically update room status when an allocation is added or removed.